# BackyardPay

A Mobile Application for Backyard Rental Management and Payment Recordkeeping

Walter Sisulu University

Department of Business and Application Development

Application Technology IV (APT 41P2)

Student: Mbongiseni Nkambule

Student number: 231064977

Project submission: 12 November 2026

## Project links

GitHub: https://github.com/nkambuleinnocent52-Ghost/Mbongiseni-Nkambule-BackyardPay

Deployed application: https://quiet-pastelito-19fa50.netlify.app

# 1. Context and background

## The rental-record problem

Small backyard rentals in East London, Eastern Cape may use cash, bank transfers, notebooks and messages to record rent. These separate records make it difficult to identify the tenant and room, the month covered, whether money was actually received and the balance still due. A tenant may have bank proof but no shared record showing how the landlord applied the payment. A payment reported by a tenant must therefore be distinguished from a payment confirmed by the landlord.

## Target users

BackyardPay serves landlords managing a small set of properties and rooms, and the tenants renting those rooms. The landlord assigns an available room and supplies a private invitation. The tenant registers against that assigned rental. Each role can access its own rental records.

## Why mobile access helps

A phone lets a tenant report money already paid, check monthly balances, retrieve receipts and describe a repair problem where it occurs. A landlord can check a payment or respond to a problem while away from a desk. The Flutter interface adapts to phone and desktop widths. An internet connection is needed to access shared account records (Flutter, n.d.).

## Scope

The responsive Flutter/Dart application is hosted on Netlify. Supabase supplies authentication and PostgreSQL storage. BackyardPay records payments made by cash, EFT or other supported methods; it does not transfer money or verify bank transactions automatically. Bank proof shows a banking transaction, while the application receipt records the landlord’s acknowledgement and the rental allocation. Security deposits are accounted for separately from rent.

Monthly rent is due on the 1st. Where selected, first-month rent is prorated using actual calendar days including move-in day. The application sets a ten-calendar-day deposit deadline. Rental agreements and applicable South African rental law govern the parties’ obligations (Republic of South Africa, 1999).

# 2. Goal and objectives

Goal: provide a clear, shared and traceable record of backyard rentals, rent, deposits and reported problems.

1. Register users and securely connect tenants to rooms already assigned by the landlord.

2. Record property details, room availability, room rates, deposits and actual move-in dates.

3. Calculate first-month charges, confirmed monthly balances and proposed allocations of advance rent.

4. Separate tenant reports from landlord-confirmed payments and produce retrievable receipts.

5. Support issue reports, responses, status tracking and notifications linked to the relevant record.

## Input → processing → output

Inputs: property address, room and rental terms; tenant contact and move-in details; payment amount, date, method, reference and purpose; issue location and description.

Processing: check availability and permissions; calculate rent in cents; propose allocations across months; confirm money received; maintain separate rent/deposit records.

Outputs: role dashboards, monthly rental accounts, pending/confirmed/rejected payment histories, receipts, room availability, issue status and notifications.

# 3.1 Access and tenant navigation

Use the menu to move between Overview, Payments, Notifications and Account. On a phone, open the compact navigation menu. Screenshots show the previous desktop layout; steps describe version 1.5.0.

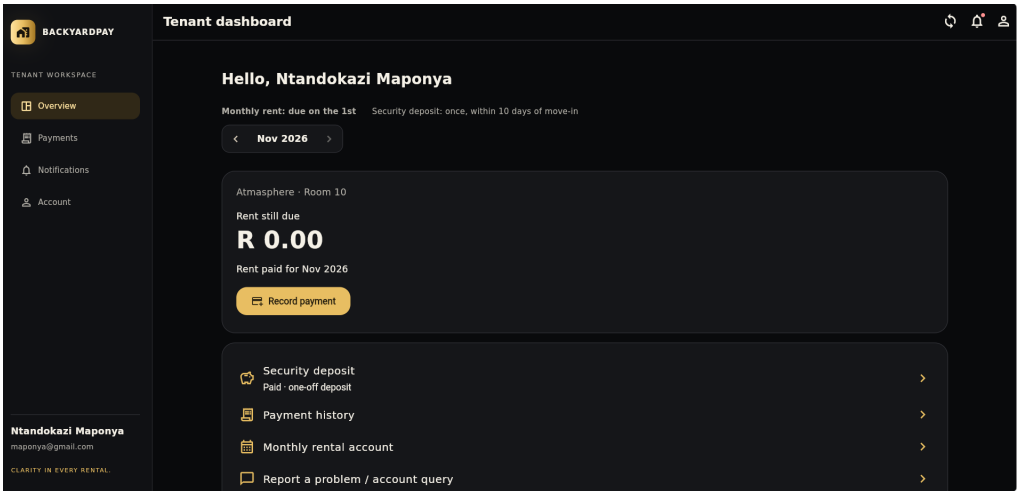



Figure 1. Tenant dashboard: selected month, rent still due and rental links.

1. Open the application and sign in with your email and password. New tenants first use the landlord’s private rental invitation. Check the property, room and selected month on the dashboard.

2. Open Payment history or Monthly rental account to inspect a specific month. Check each month separately for outstanding rent.

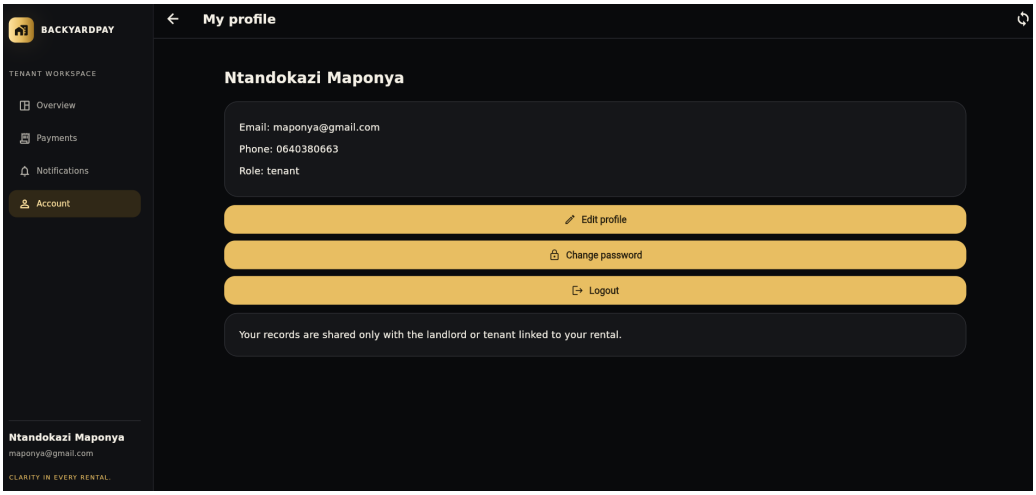



Figure 2. Account screen: contact details and account actions.

3. Use Account to review or edit your profile and change your password. In version 1.5.0, the top-right logout icon opens a nearby Cancel/Logout confirmation on each authenticated page.

4. A valid session normally survives closing and reopening the browser. Explicit logout or cleared browser storage requires another login. Optional Login ID uses the same password and requires separate server setup; email login remains available.

# 3.2 Report rent and review allocation

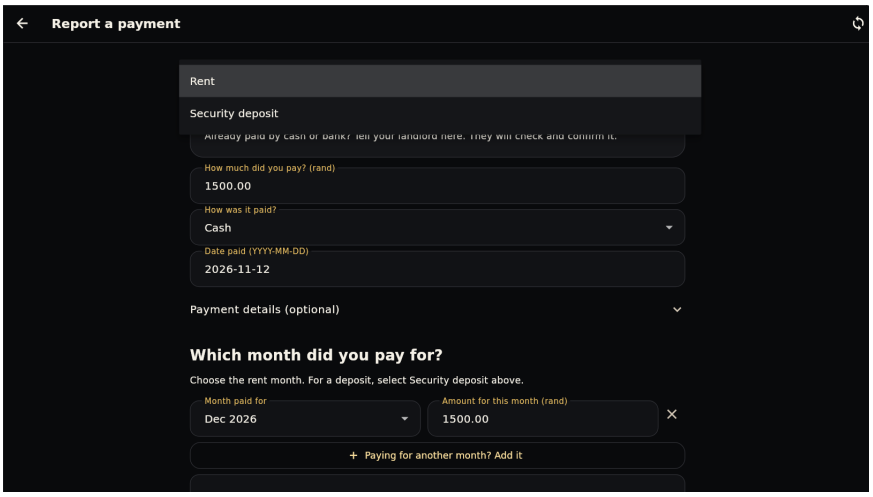



Figure 3. Tenant payment form: purpose, amount, method, payment date and reference.

1. Pay through the usual cash or banking arrangement first. Choose Record payment, select Rent and enter the actual amount, method and date paid. Enter the banking reference when required. The payment date records a past transaction; it does not change the monthly due day.

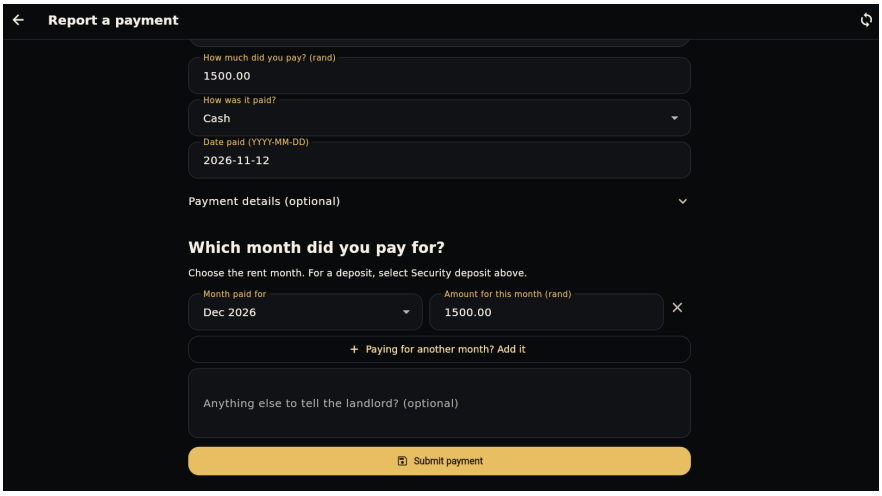



Figure 4. Rent-month allocation and submission controls.

2. Review which months the payment covers before submitting. Version 1.5.0 proposes a split using the amount still owed, beginning with the selected month. You can inspect the proposal before sending it.

3. Example: R2,000 against R1,200 due allocates R1,200 now and R800 to the next month. If next month costs R1,500, R700 remains after confirmation. Deposit money is excluded from this calculation.

4. Submit the report. It stays Pending until the landlord checks actual receipt. The landlord confirms the submitted allocation or rejects the report with an explanation; pending money does not reduce confirmed rent owed.

# 3.3 Monthly account and deposit

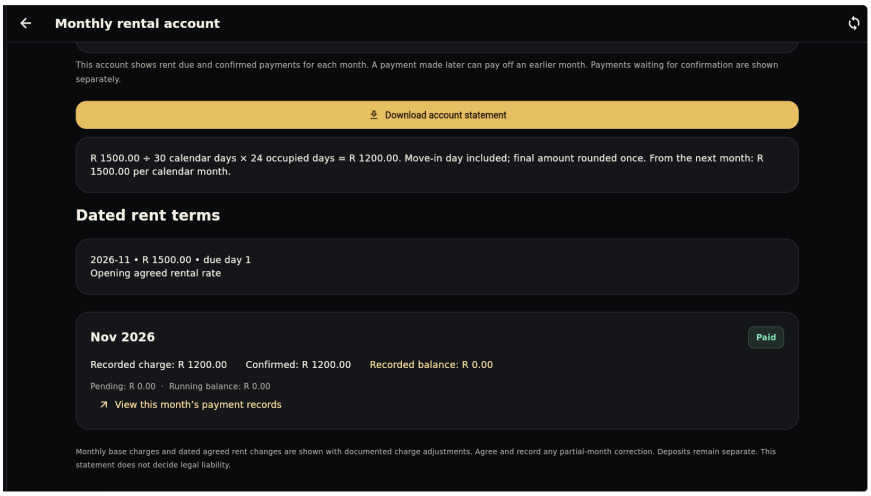



Figure 5. Monthly rental account with dated rental terms and monthly balances.

1. Open Monthly rental account. Select the month and compare the charge, confirmed rent received and amount still due. Use Payment history to open a payment and its available confirmed receipt.

2. A future rent-change start month identifies when a new amount begins. It does not move the due day from the 1st. Version 1.5.0 shows the current/new amount and the affected tenant. Earlier charges remain unchanged; the agreement and applicable notice requirements still matter.

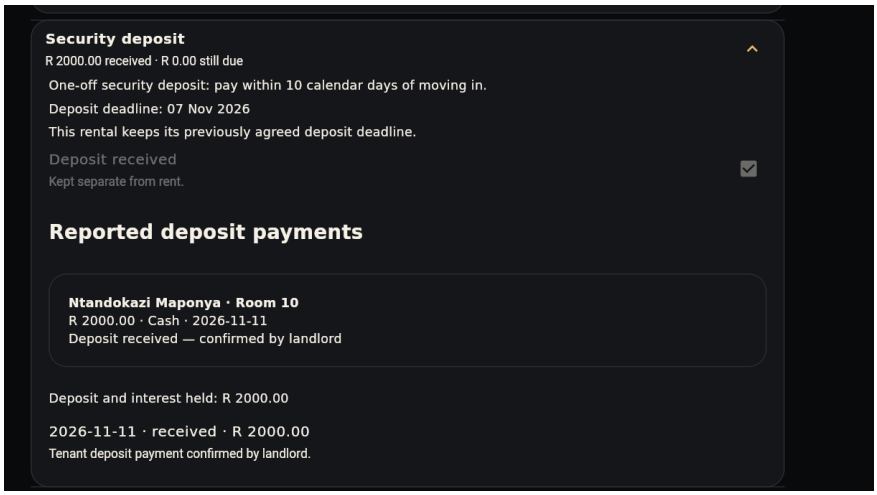



Figure 6. Security deposit record, deadline and reported deposit payments.

3. When reporting deposit money, choose Security deposit. Review the separate deposit record and landlord confirmation. A rent overpayment is not automatically a deposit payment.

4. The ten-day deadline is the project policy. Deposit returns and supported deductions are recorded separately. The deposit is not automatically forfeited simply because a tenant leaves (Republic of South Africa, 1999).

# 3.4 Landlord: rooms and onboarding

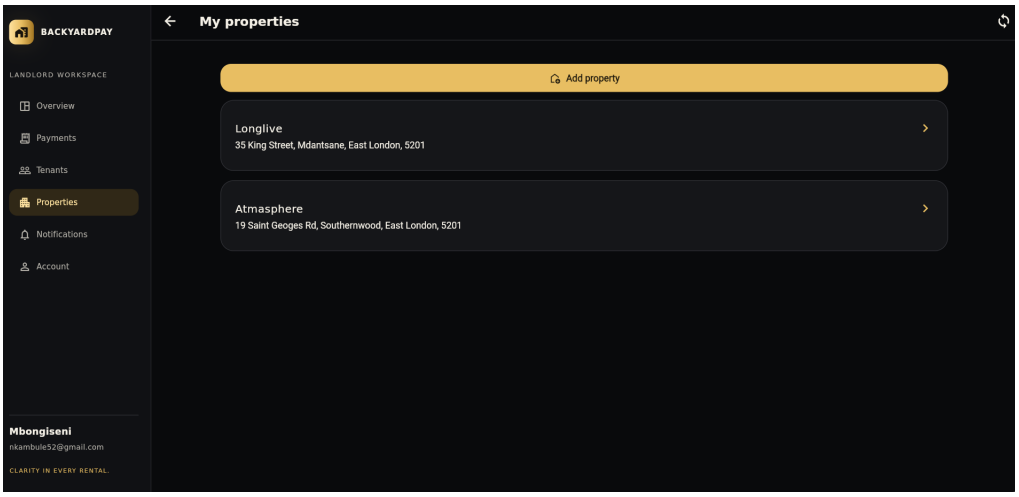



Figure 7. Landlord property list with property names and supplied addresses.

1. Open Properties and choose Add property. Enter the full address, room count, default monthly rent and separate deposit. Use room-specific rates for rooms with different prices. Version 1.5.0 stores shared property terms once and uses them for new rental records.

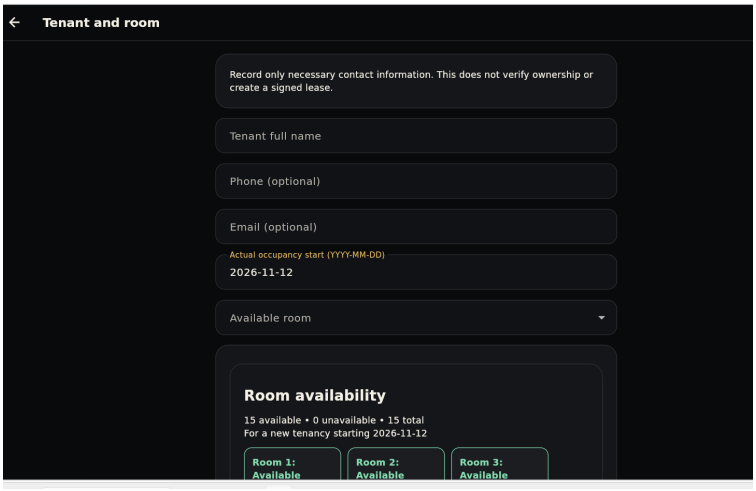



Figure 8. Add tenant record: tenant details, move-in date and available rooms.

2. Open Add tenant record, select the property and an available room, and enter the tenant's contact details and actual move-in date. Check the itemised first rent and deposit before saving.

3. Share that rental's private invitation with its tenant. The tenant creates their own account; a public property code cannot assign a room. The move-in date records occupancy rather than the day an account was created.

4. In version 1.5.0, open the property address in Google Maps to search the supplied address. GPS capture is no longer required. Check the address shown in Maps before using directions.

# 3.5 Review, notifications and problems

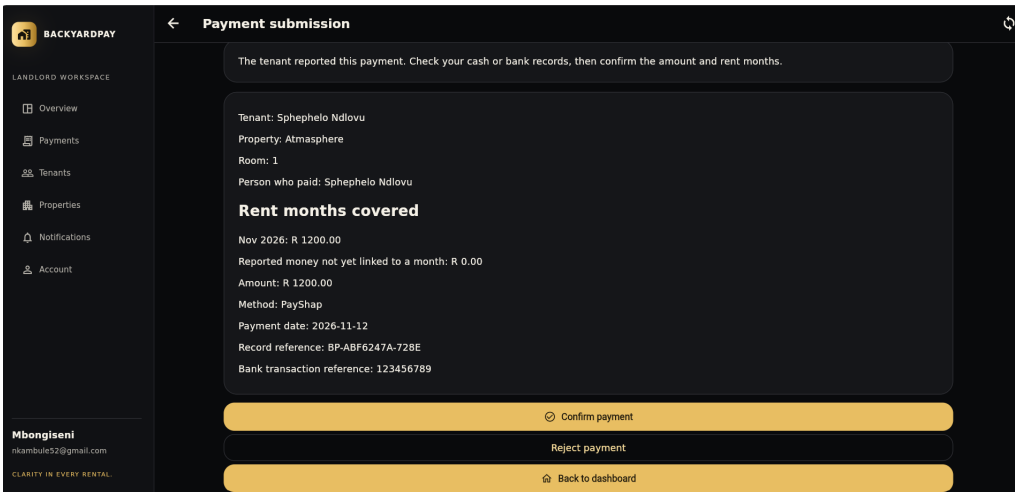



Figure 9. Landlord payment details and receipt-check actions.

1. Open a pending rent or deposit report. Check the tenant, room, purpose, amount, reference and actual money received. Confirm only after checking; otherwise reject with an explanation. A report alone is not proof of receipt.

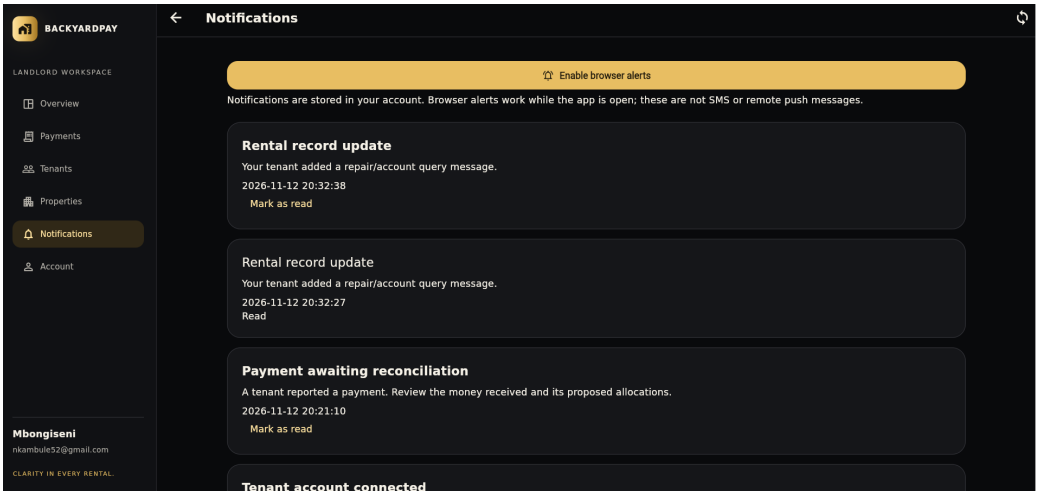



Figure 10. Notifications and the Mark as read action.

2. In version 1.5.0, new notifications identify the tenant/room and open the relevant payment or issue. Older notifications may open the payment or issue list. Mark as read clears unread status; it does not confirm a payment.

3. Tenant: open Report a problem, choose the category/location and describe the issue. Send it once. Open its conversation for follow-up. Landlord: acknowledge receipt, respond and update Submitted → Acknowledged → In progress → Resolved.

4. In-app notices are stored with the account. Browser alerts require permission and work while the app is open; they are not SMS or remote push. Review the record itself for the current status.

# Appendix A.1 Structure and roles

Selected source is taken from version 1.5.0, commit cd00a9c. The complete source code and SQL are available in the repository.

## Organised folder structure

lib/main.dart — application and Supabase startup

lib/app.dart — session handling, theme and routing

lib/screens/auth/, landlord/, tenant/, shared/ — role screens and navigation

lib/screens/properties/, payments/, rentals/ — property, money and rental workflows

lib/models/ — typed users, properties, tenancies and payments

lib/data/ — shared state, cloud gateway and accounting calculations

lib/services/ and lib/ui/ — receipts, maps, alerts and visual components

supabase/migrations/ and supabase/tests/ — database rules and database checks

test/ and tool/ — Dart checks, browser checks, builds and release packaging

Source: lib/screens/shared/dashboard.dart, lines 13–24.

```
13  // Select the page from the signed-in account's role.
14  class DashboardScreen extends StatelessWidget {
15    const DashboardScreen({super.key});
16    @override
17    Widget build(BuildContext context) {
18      final user = StoreScope.of(context).currentUser;
19      if (user == null) return const Scaffold(body: SizedBox.shrink());
20      return user.role == Role.landlord
21          ? const LandlordDashboardScreen()
22          : const TenantDashboardScreen();
23    }
24  }
```

Purpose: choose the dashboard from the authenticated account’s role. StoreScope supplies shared state. This screen selection controls presentation; database permissions must separately prevent unauthorised access.

# Appendix A.2 Calendar-day rent

Source: lib/data/research_ledger.dart, lines 69–74.

```
69  /// Calendar-day billing, including the move-in day. Round the final cents only.
70  int daysInMonth(DateTime date) => DateTime(date.year, date.month + 1, 0).day;
71  int proratedFirstCharge(int monthlyCents, DateTime start) {
72    final days = daysInMonth(start), occupied = days - start.day + 1;
73    return (monthlyCents * occupied * 2 + days) ~/ (2 * days);
74  }
```

Purpose: calculate a first partial month using actual month length. The occupied-day count includes move-in day. Amounts use integer cents, and rounding happens once on the final charge rather than on a daily amount.

## Worked example

Monthly rent R1,500; November has 30 days; move-in on 7 November gives 24 occupied days. First rent = R1,500 × 24 ÷ 30 = R1,200. Normal recurring rent remains R1,500 due on the 1st from the following month. A different move-in day produces a different charge.

## Why this matters

Using actual days avoids treating every month as thirty days. Integer cents avoid accumulating floating-point currency errors. This calculation does not record a payment: it creates the charge against which confirmed rent is compared. A property can instead use its selected full-first-month billing rule.

## Rent changes and recorded history

A later agreed rate begins in its effective future month. The previous rate remains in dated history so earlier monthly accounts are not rewritten. Room size and agreement terms can justify different rates; the application does not assume every room must have the same amount or every change applies to an entire year.

# Appendix A.3 Proposed rent split

Source: lib/data/rent_payment_plan.dart, lines 5–27.

```
 5  List<PaymentAllocation> proposeRentPayment({
 6    required int amountCents,
 7    required String firstPeriod,
 8    required Iterable<String> allowedMonths,
 9    required int Function(String) amountStillToReport,
10  }) {
11    var remaining = amountCents;
12    final months =
13        allowedMonths.where((m) => m.compareTo(firstPeriod) >= 0).toSet().toList()
14          ..sort();
15    final result = <PaymentAllocation>[];
16    for (final month in months) {
17      if (remaining <= 0) break;
18      final available = amountStillToReport(month).clamp(0, 2147483647);
19      final part = remaining < available ? remaining : available;
20      if (part > 0) {
21        result.add(PaymentAllocation(month, part));
22        remaining -= part;
23      }
24    }
25    return result;
26  }
```

Purpose: propose how one rent payment is distributed, starting at the tenant’s selected month. Allowed months are filtered, sorted and de-duplicated. Each month receives at most the remaining amount needed before money moves to a later month.

For 200,000 cents against 120,000 cents due now, the proposal uses 120,000 cents now and 80,000 cents in the next month. The equivalent rand figures are R2,000, R1,200 and R800.

The function returns a proposal; it does not confirm receipt or mix in a security deposit. The tenant checks the split before submission. The server validates the saved proposal and protects it during landlord review. If the available month range cannot absorb the payment, validation must require a complete valid allocation before saving.

# Appendix A.4 Availability and validation

Source: lib/data/room_availability.dart, lines 5–26.

```
 5  List<int> availableRoomsFor(
 6    RentalProperty property,
 7    Iterable<Tenancy> tenancies,
 8    DateTime moveIn, {
 9    bool actualHandover = true,
10  }) {
11    final day = DateTime(moveIn.year, moveIn.month, moveIn.day);
12    final blocked = <int>{};
13    for (final t in tenancies.where((t) => t.propertyId == property.id)) {
14      final end = actualHandover ? t.endedOn : t.leaseEnd;
15      if (end == null || !DateTime(end.year, end.month, end.day).isBefore(day)) {
16        final room = int.tryParse(t.room);
17        if (room != null) blocked.add(room);
18      }
19    }
20    return [
21      for (var room = 1; room <= property.rooms; room++)
22        if (!blocked.contains(room)) room,
23    ];
24  }
```

Purpose: exclude rooms whose previous rental has not actually ended before the proposed move-in day. The default uses recorded room handover, rather than assuming an agreed lease-end date means the room is empty. Availability is checked again when saving.

## Validation and evidence

The version 1.5.0 release records 105 passing Flutter tests and analysis of 78 Dart files with no issues. Database checks cover migrations and access rules. Chromium workflow checks cover payment allocation, deposits, issue status, notification targets and session behaviour.

# References

Flutter (n.d.) Flutter documentation. Available at: https://docs.flutter.dev/ (Accessed: 13 November 2026).

Netlify (n.d.) Deploy overview. Available at: https://docs.netlify.com/deploy/overview/ (Accessed: 13 November 2026).

Nkambule, M. (2026) BackyardPay source repository, version 1.5.0, commit cd00a9c. Available at: https://github.com/nkambuleinnocent52-Ghost/Mbongiseni-Nkambule-BackyardPay (Accessed: 13 November 2026).

Republic of South Africa (1999) Rental Housing Act 50 of 1999. Available at: https://www.gov.za/documents/rental-housing-act (Accessed: 13 November 2026).

Supabase (n.d.) Auth and Row Level Security documentation. Available at: https://supabase.com/docs/guides/auth and https://supabase.com/docs/guides/database/postgres/row-level-security (Accessed: 13 November 2026).

Walter Sisulu University (2026) Application Technology IV project guideline 2026. Supplied module document.

## Screenshot source

BackyardPay application screenshots from TENANT N LANDLORD DASHBOARD.docx.